

Página de Análisis de Datos Genéticos de Antropología Molecular

Alessandro López-Hernández

2026-01-04

Table of contents

1	Bienvenida	3
1.1	Instalación de R y RStudio	3
2	R para principiantes	5
3	Vectores	8
4	Matrices	9
5	Data frames	11
6	Listas	13
7	Trabajos en el Laboratorio I	15
7.1	Descarga de datos desde GenBank	15
8	Alineamiento de secuencias	19
9	Ejercicio	21
10	Análisis de Distancias Genéticas	22
11	Algoritmos de construcción de árboles filogenéticos	26
12	Ejercicio 2:	29
13	Referencias	30
14	Summary	31
	References	32

1 Bienvenida

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

1.1 Instalación de R y RStudio

Para comenzar, necesitamos instalar R y RStudio. R es el lenguaje de programación, mientras que RStudio es un entorno de desarrollo integrado (IDE) que facilita el uso de R. 1. **Instalar R:** Puedes descargar R desde el sitio web del [Proyecto R](#). Sigue las instrucciones para tu sistema operativo (Windows, macOS o Linux). 2. **Instalar RStudio:** Después de instalar R, puedes descargar RStudio desde el sitio web de [RStudio](#). Elige la versión gratuita (RStudio Desktop) y sigue las instrucciones de instalación.

Una vez que hayas instalado R y RStudio, abre RStudio para comenzar a trabajar con tus datos de secuencias de ADN mitocondrial. En la próxima sección, aprenderemos cómo cargar y manipular estos datos en R.

¡Bienvenidos al **Laboratorio de Antropología Molecular** de la Facultad de Ciencias, UNAM!.



Figure 1.1: Miembros del Laboratorio de AM en Teotihuacán. Créditos a Alessandro López-Hernandez, 2023.

2 R para principiantes

Nota: esta sección está diseñada para estudiantes que no tienen experiencia previa en programación, en especial a R. Si ya tienes experiencia en programación, puedes saltar esta sección.

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

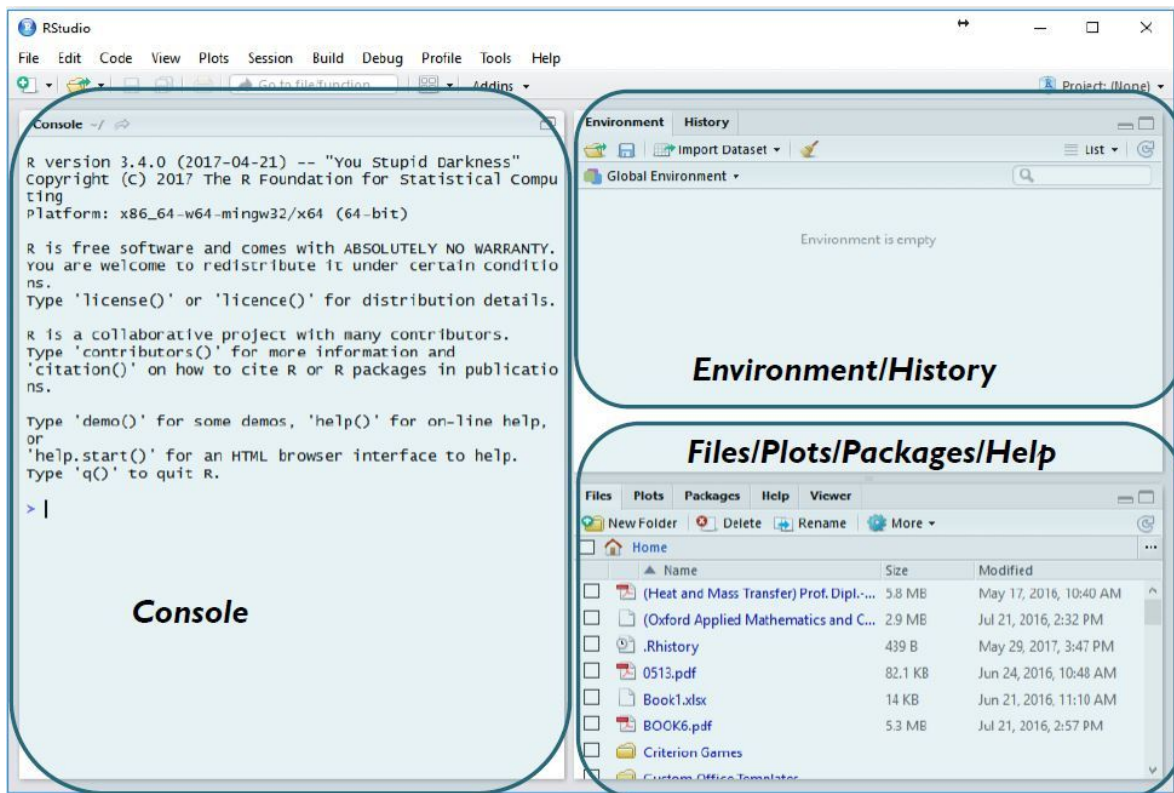


Figure 2.1: Interfaz de RStudio, un entorno de desarrollo integrado para R. Créditos a “Geeks-ForGeeks”

Arriba pueden observar lo que es una interfaz de RStudio. La ventana de la izquierda es la consola. La consola es donde se ejecutan las instrucciones que escribimos en el script. Por ejemplo, si queremos cargar un paquete, tenemos que escribir `library(nombre_del_paquete)` en el script y luego ejecutar esa línea de código para que se cargue el paquete en la consola.

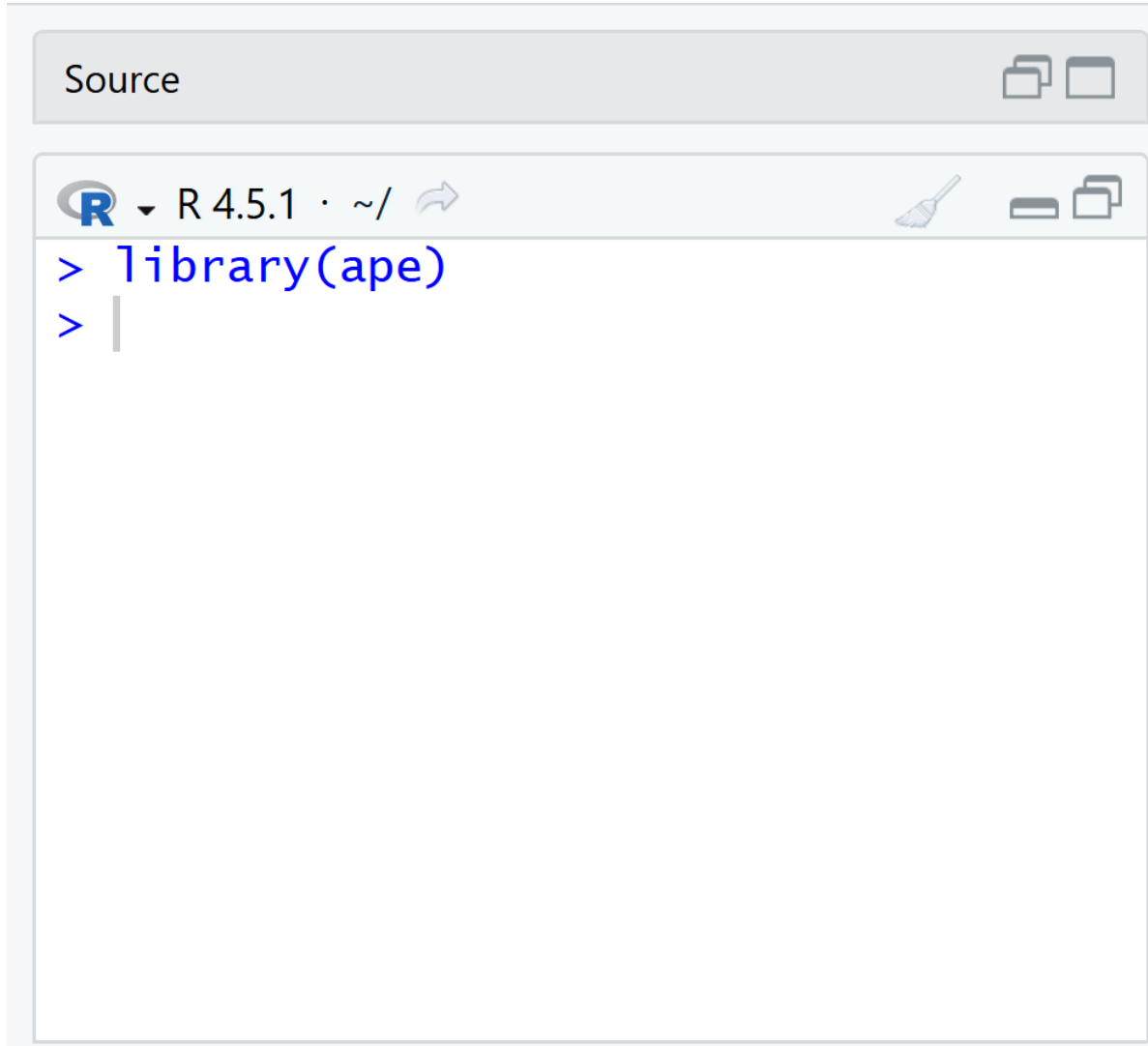


Figure 2.2: Ejecución de una instrucción en RStudio.

Como notarás, no se ejecuta nada más. !Y eso está bien!. No prestemos por ahora atención sobre los mensajes que aparecen en la consola. Lo importante es que se ejecute la instrucción que escribimos en el script. **En este caso, lo que hicimos fue cargar el paquete ape, que es un paquete muy útil para trabajar con datos de secuencias de ADN.**

A partir de esta sección, ya no verás imágenes de la consola de RStudio ejecutándose, sino que ahora verás el código tal cuál debe escribirse en tu consola. Esto es para que puedas copiar y pegar el código directamente en tu consola de RStudio y así practicar lo que estás aprendiendo.

Lo siguiente que haremos es generar datos para que R pueda leerlos. Los datos en R se clasifican en diferentes tipos: *vectores*, *matrices*, *data frames* y *listas*.

3 Vectores

Los vectores son la forma más básica de datos en R. Un vector es *una secuencia de elementos del mismo tipo*. Por ejemplo, un vector de números enteros, un vector de caracteres, etc. Para crear un vector en R, usamos la función `c()`, que significa “combine” o “concatenate”. Por ejemplo, para crear un vector de números enteros del 1 al 5, escribimos:

```
mi_vector <- c(1, 2, 3, 4, 5)
```

En este caso, `mi_vector` es un vector que contiene los números del 1 al 5. Podemos acceder a los elementos de un vector usando índices. Por ejemplo, para acceder al tercer elemento de `mi_vector`, escribimos:

```
mi_vector[3]
```

```
[1] 3
```

Esto nos devolverá el número 3, que es el tercer elemento del vector. También podemos realizar operaciones con vectores. Por ejemplo, si queremos sumar 2 a cada elemento del vector, escribimos:

```
mi_vector + 2
```

```
[1] 3 4 5 6 7
```

Esto nos devolverá un nuevo vector con los valores 3, 4, 5, 6 y 7.

Nota: los vectores son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en un vector para facilitar su análisis. Por ejemplo:

```
secuencias <- c("ATCG", "GCTA", "TACG", "CGTA")
```

En este caso, `secuencias` es un vector que contiene las secuencias de ADN. Cada elemento del vector es una secuencia de ADN representada como una cadena de caracteres. Podemos acceder a cada secuencia usando índices, por ejemplo:

4 Matrices

Las matrices son una extensión de los vectores. Una matriz es *una colección de elementos del mismo tipo organizados en filas y columnas*. Para crear una matriz en R, usamos la función `matrix()`. Por ejemplo, para crear una matriz de 2 filas y 3 columnas con los números del 1 al 6, escribimos:

```
mi_matriz <- matrix(1:6, nrow = 2, ncol = 3)
```

En este caso, `mi_matriz` es una matriz que contiene los números del 1 al 6 organizados en 2 filas y 3 columnas. Podemos acceder a los elementos de una matriz usando índices de fila y columna. Por ejemplo, para acceder al elemento en la primera fila y segunda columna, escribimos:

```
mi_matriz[1, 2]
```

```
[1] 3
```

Esto nos devolverá el número 4, que es el elemento en la primera fila y segunda columna de la matriz. También podemos realizar operaciones con matrices. Por ejemplo, si queremos sumar 2 a cada elemento de la matriz, escribimos:

```
mi_matriz + 2
```

```
      [,1] [,2] [,3]  
[1,]    3    5    7  
[2,]    4    6    8
```

Esto nos devolverá una nueva matriz con los valores 3, 4, 5, 6, 7 y 8.

Nota: las matrices son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en una matriz para facilitar su análisis. Por ejemplo:

```
secuencias_matriz <- matrix(c("ATCG", "GCTA", "TACG", "CGTA"), nrow = 2, ncol = 2)
secuencias_matriz
```

```
      [,1] [,2]
[1,] "ATCG" "TACG"
[2,] "GCTA" "CGTA"
```

En este caso, `secuencias_matriz` es una matriz que contiene las secuencias de ADN organizadas en 2 filas y 2 columnas. Cada elemento de la matriz es una secuencia de ADN representada como una cadena de caracteres.

5 Data frames

Los data frames son una estructura de datos muy común en R. Un data frame es *una tabla de datos que puede contener diferentes tipos de variables*. Para crear un data frame en R, usamos la función `data.frame()`. Por ejemplo, para crear un data frame con información sobre secuencias de ADN, escribimos:

```
secuencias_df <- data.frame(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_df` es un data frame que contiene información sobre las secuencias de ADN. La primera columna `id` contiene los identificadores de las secuencias, la segunda columna `secuencia` contiene las secuencias de ADN y la tercera columna `longitud` contiene la longitud de cada secuencia. Podemos acceder a las columnas de un data frame usando el operador `$`. Por ejemplo, para acceder a la columna `secuencia`, escribimos:

```
secuencias_df$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a las filas de un data frame usando índices. Por ejemplo, para acceder a la primera fila del data frame, escribimos: `mtcsecuencias_df[1, ]` Esto nos devolverá la primera fila del data frame, que contiene la información sobre la secuencia `seq1`. También podemos realizar operaciones con data frames. Por ejemplo, si queremos agregar una nueva columna que contenga el número de A's en cada secuencia, escribimos: `mtcsecuencias_df$num_A <- sapply(secuencias_df$secuencia, function(x) sum(str_count(x, "A")))` Esto nos agregará una nueva columna `num_A` al data frame que contiene el número de A's en cada secuencia de ADN.

Nota: los data frames son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar la información sobre las secuencias en un data frame para facilitar su análisis. Por ejemplo, podemos

agregar más columnas al data frame para incluir información adicional sobre las secuencias, como la especie a la que pertenecen, la ubicación geográfica donde se encontraron, etc. Esto nos permitirá realizar análisis más complejos y obtener insights más profundos sobre nuestras secuencias de ADN.

6 Listas

Las listas son una estructura de datos muy flexible en R. Una lista es *una colección de elementos que pueden ser de diferentes tipos*. Para crear una lista en R, usamos la función `list()`. Por ejemplo, para crear una lista que contenga información sobre secuencias de ADN, escribimos:

```
secuencias_lista <- list(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_lista` es una lista que contiene información sobre las secuencias de ADN. La lista tiene tres elementos: `id`, `secuencia` y `longitud`. Cada elemento de la lista es un vector que contiene la información correspondiente. Podemos acceder a los elementos de una lista usando el operador `$`. Por ejemplo, para acceder al elemento `secuencia`, escribimos:

```
secuencias_lista$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a los elementos de una lista usando índices. Por ejemplo, para acceder al primer elemento de la lista, escribimos:

```
secuencias_lista[[1]]
```

```
[1] "seq1" "seq2" "seq3" "seq4"
```

Esto nos devolverá el vector `id` que contiene los identificadores de las secuencias. Las listas son muy útiles cuando queremos almacenar información que no necesariamente tiene la misma estructura, como por ejemplo, una lista de secuencias de ADN donde cada secuencia puede tener una longitud diferente. Por ejemplo, podemos crear una lista de secuencias de ADN donde cada secuencia es un vector de caracteres de diferente longitud:

```
secuencias_lista <- list(  
  seq1 = c("A", "T", "C", "G"),  
  seq2 = c("G", "C", "T", "A"),  
  seq3 = c("T", "A", "C"),  
  seq4 = c("C", "G")  
)
```

En este caso, **secuencias_lista** es una lista que contiene cuatro elementos, cada uno de los cuales es un vector de caracteres que representa una secuencia de ADN. Cada secuencia tiene una longitud diferente, lo que demuestra la flexibilidad de las listas en R.

Con eso concluye esta sección introductoria sobre R para principiantes. En las siguientes secciones, aprenderemos a usar R para realizar análisis más complejos con nuestros datos de secuencias de ADN mitocondrial.

7 Trabajos en el Laboratorio I

En el laboratorio de Antropología Molecular, es común trabajar con datos de secuencias de ADN mitocondrial (mtDNA). En esta sección, aprenderemos a descargar datos desde GenBank y a leerlos en R para su posterior análisis.

Nota: Asegúrate de tener listos tus archivos de secuencias en formato FASTA antes de comenzar. Si no los tienes, puedes continuar siguiendo las instrucciones que daré a continuación.

7.1 Descarga de datos desde GenBank

GenBank es una base de datos pública de secuencias de ADN mantenida por el NCBI [National Center for Biotechnology Information; Sayers et al. (2022)]. Para descargar datos de GenBank, puedes utilizar la función `rentrez` en R o hacerlo manualmente a través de la página web del NCBI. Para descargar los datos manualmente, naveguen a la página de NCBI y seleccionen el formato FASTA como se muestra a continuación:

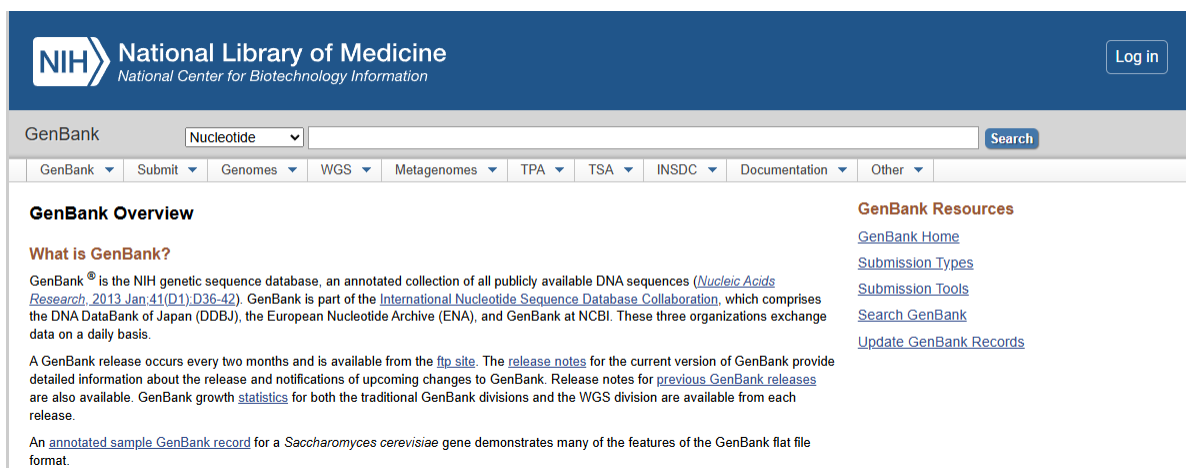


Figure 7.1: Página de inicio de NCBI. Las opciones por default se muestran en la imagen

Lo primero que debemos hacer es buscar una secuencia de interés. Para esto, podemos utilizar el número de acceso (accession number) o el nombre del organismo. En este caso, decidí usar solo las secuencias de mtDNA de la región control, que es una región comúnmente utilizada en

estudios de genética de poblaciones. Para esto, busqué “Human mitochondrial DNA control region” en la barra de búsqueda y seleccioné las secuencias que me interesaban.

Nucleotide

Human mitochondrial DNA control region

Create alertAdvanced

Summary200 per pageSort by Default orderSend to:

Items: 1 to 200 of 11359

<< First < Prev Page 1 of 57 Next > Last >>

Filters activated: Sequence length from 1 to 400. Clear all

☐

[Human mitochondrial DNA control region..genotype AK1110](#)

1. 377 bp linear DNA

Accession: U37731.1 GI: 1022848

[PubMed](#) [Taxonomy](#)

[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AK27](#)

2. 377 bp linear DNA

Accession: U37730.1 GI: 1022847

[PubMed](#) [Taxonomy](#)

[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW222](#)

3. 377 bp linear DNA

Accession: U37733.1 GI: 1022850

[PubMed](#) [Taxonomy](#)

[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW218](#)

4. 377 bp linear DNA

Accession: U37732.1 GI: 1022849

[PubMed](#) [Taxonomy](#)

[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype HK3992](#)

5. 377 bp linear DNA

Accession: U37737.1 GI: 1022854

[PubMed](#) [Taxonomy](#)

[GenBank](#) [FASTA](#) [Graphics](#)

clear

Figure 7.2: Captura de pantalla de la búsqueda en NCBI. En la barra de búsqueda se muestra “HVR1 mtDNA”

Una vez que tenga identificadas las secuencias de interés, puedo descargarlas directamente, sin embargo, para este ejercicio, usaremos el paquete `ape` para descargar las secuencias directamente desde R, lo que nos permitirá automatizar el proceso y evitar errores manuales. Para ello, debo identificar el número de acceso de la secuencia que quiero descargar. En este caso, usaré la secuencia con el número de acceso “U37731.1”, que corresponde a una secuencia de mtDNA humana de la región control.

```
options(warn = -1) # oculta warnings

# Cargamos el paquete "ape"
library(ape)
# Descargamos la secuencia de interés utilizando su número de acceso
secuencia_1 <- read.GenBank("U37731.1")
# Visualizamos la secuencia descargada
print(secuencia_1)
```

1 DNA sequence in binary format stored in a list.

Sequence length: 377

Label:
U37731.1

Base composition:

a	c	g	t
0.322	0.324	0.128	0.226

(Total: 377 bases)

Sin embargo, es necesario que se descarguen al menos un número de secuencias para que el código funcione correctamente. Para esto, podemos descargar varias secuencias utilizando sus números de acceso. Por ejemplo, podemos descargar las siguientes secuencias:

```
# Descargamos varias secuencias utilizando sus números de acceso
secuencias <- read.GenBank(c("U37731.1", "U37730.1", "U37733.1", "U37737.1", "U37752.1",
                             "U37751.1", "U37739.1", "U37743.1"))
# Visualizamos las secuencias descargadas
print(secuencias)
```

8 DNA sequences in binary format stored in a list.

Mean sequence length: 377.125

Shortest sequence: 377
Longest sequence: 378

Labels:
U37731.1
U37730.1
U37733.1
U37737.1
U37752.1
U37751.1
...

Base composition:
a c g t
0.320 0.336 0.125 0.219
(Total: 3.02 kb)

Sin embargo, es importante mencionar que hay una secuencia que no tiene el mismo tamaño que las demas, lo que puede causar problemas en un analisis posterior. Para evitar esto, es recomendable descargar secuencias que tengan el mismo tamaño o recortarlas para que tengan el mismo tamaño antes de analizarlas o en su defecto, hacer un alineamiento de secuencias para que todas tengan el mismo tamaño. Aquí haremos un alineamiento de secuencias utilizando el paquete `msa` para asegurarnos de que todas las secuencias tengan el mismo tamaño antes de analizarlas.

8 Alineamiento de secuencias

El alineamiento requiere de un paquete adicional llamado `msa`, que se debe de instalar antes de continuar. Para instalarlo, puedes usar el siguiente código:

```
# Instalación de Bioconductor
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
# Instalación del paquete msa
BiocManager::install("msa")
```

Bioconductor es un proyecto de software de código abierto que proporciona herramientas para el análisis de datos genómicos y biológicos. El paquete `msa` es una herramienta de alineamiento de secuencias que permite alinear múltiples secuencias de ADN, ARN o proteínas. Para alinear las secuencias descargadas, podemos usar la función `msa` del paquete `msa`.

****Nota:** Para que el alineamiento funcione correctamente, el paquete `msa` requiere que las secuencias estén en un formato “AAStringSet” o “DNAStrngSet”. Sin embargo, el paquete `ape` devuelve las secuencias en un formato “DNABin”. Por lo tanto, es necesario transformar las secuencias descargadas en un formato compatible con el paquete `msa` antes de alinearlas. Para esto, podemos usar la función `as.DNABin` del paquete `ape` para transformar las secuencias en un formato compatible con el paquete `msa`:

```
suppressPackageStartupMessages(library(msa))
suppressPackageStartupMessages(library(Biostrings))

# Descargamos las secuencias utilizando sus números de acceso
secuencias_crudas <- read.GenBank(c("U37731.1", "U37730.1", "U37733.1", "U37737.1", "U37752.1",
                                   "U37751.1", "U37739.1", "U37743.1"))

# Este paso es esencial para convertir las secuencias al formato que el paquete msa requiere
secuencias_letras <- as.character(secuencias_crudas) # Transformamos las secuencias en un formato compatible

# Esto convierte c("A", "T", "G") en "ATG" para cada individuo
secuencias_texto <- sapply(secuencias_letras, paste, collapse = "")

# Convertimos el texto limpio al formato que msa exige
```

```

secuencias_biostrings <- DNASTringSet(secuencias_texto)

# Alineamos las secuencias utilizando la función msa
alineamiento <- msa(secuencias_biostrings, method = "Muscle")

# Visualizamos el alineamiento
print(alineamiento)

```

MUSCLE 3.8.31

Call:

```
msa(secuencias_biostrings, method = "Muscle")
```

MsaDNAMultipleAlignment with 8 rows and 378 columns

	aln	names
[1]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGMT-AGGGGTCCCTTGAC	U37733.1
[2]	TTCTTTTATGGGGMAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCATGAC	U37731.1
[3]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37730.1
[4]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37752.1
[5]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37737.1
[6]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37739.1
[7]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37743.1
[8]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGATCAGGGGTCCCTTGAC	U37751.1
Con	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	Consensus

¡Felicidades! Has generado un alineamiento de secuencias utilizando el paquete `msa`. Este alineamiento es esencial para realizar análisis posteriores, como la construcción de árboles filogenéticos o el cálculo de distancias genéticas entre las secuencias. En la próxima sección, aprenderemos a construir un árbol filogenético utilizando el alineamiento generado.

9 Ejercicio

1. Descarga al menos entre 10 a 30 secuencias de de DNA diferentes organismos (pueden ser humanos o de otras especies) utilizando el paquete `ape` o manualmente desde GenBank. **Nota: Asegúrate de que las secuencias descargadas sean de la misma región del ADN para evitar problemas en el análisis posterior. Por ejemplo, puedes descargar secuencias de la región control del mtDNA de diferentes individuos o especies. Además, evita usar secuencias mayores a 1000 pb para no sobrecargar el sistema de tu computadora.**
2. Alinea las secuencias descargadas utilizando el paquete `msa` y visualiza el alineamiento.
3. Compara el alineamiento generado con las secuencias originales para asegurarte de que todas las secuencias tengan el mismo tamaño y que el alineamiento se haya realizado correctamente.
4. Guarda el alineamiento en un archivo FASTA para su posterior análisis. Para ello, necesitas usar la función `writeXStringSet` del paquete `Biostrings` para guardar el alineamiento en un archivo FASTA. Aquí te dejo un ejemplo de cómo hacerlo:

```
# Guardamos el alineamiento en un archivo FASTA
alineamiento_texto <- as(alineamiento, "DNASTringSet")

# El archivo se va a guardar donde sea que tengas tu proyecto de R. Si quieres guardarlo en v
writeXStringSet(alineamiento_texto, filepath = "../Results/alineamiento.fasta")
```

10 Analisis de Distancias Genéticas

Una vez que tenemos nuestro alineamiento de secuencias, podemos calcular las distancias genéticas entre las secuencias para entender las relaciones evolutivas entre ellas. Para esto, podemos usar la función `dist.dna` del paquete `ape`, que calcula las distancias genéticas entre las secuencias alineadas utilizando diferentes modelos de sustitución de nucleótidos. Anteriormente, era necesario transformar el alineamiento al formato “DNABin” para que la función `dist.dna` pueda trabajar con él. Sin embargo, el paquete `msa` devuelve el alineamiento en un formato “MsaDNAMultipleAlignment”, por lo que es necesario transformar el alineamiento al formato “DNABin” antes de calcular las distancias genéticas. Para esto, podemos usar la función `as.DNABin` del paquete `ape` para transformar el alineamiento al formato “DNABin”:

```
# Transformamos el alineamiento al formato "DNABin"
alineamiento_dnabin <- as.DNABin(alineamiento)

# Calculamos las distancias genéticas entre las secuencias utilizando la función dist.dna
distancias_geneticas <- dist.dna(alineamiento_dnabin, model = "K80") # Puedes cambiar el model

# Visualizamos las distancias genéticas
print(distancias_geneticas)
```

```

          U37733.1    U37731.1    U37730.1    U37752.1    U37737.1
U37731.1 0.049988834
U37730.1 0.047081849 0.008042973
U37752.1 0.044371795 0.032909975 0.030085109
U37737.1 0.035679771 0.035571725 0.032754816 0.030029993
U37739.1 0.041481614 0.035750892 0.032909975 0.030228985 0.027228865
U37743.1 0.047278780 0.035750892 0.032909975 0.030228985 0.027228865
U37751.1 0.035750892 0.018942587 0.016195327 0.019024034 0.016178692
          U37739.1    U37743.1
U37731.1
U37730.1
U37752.1
U37737.1
U37739.1
U37743.1 0.010782089
U37751.1 0.016261596 0.016261596
```

En este punto, estas funciones te serán conocidas puesto que has trabajado con ellas en el Laboratorio de Antropología Molecular. La distancia genética usada en este modelo es el modelo de Kimura de 2 parámetros (K80), que asume que las sustituciones de nucleótidos pueden ser transiciones (cambio entre purinas o entre pirimidinas) o transversiones (cambio entre purina y pirimidina), y que las tasas de sustitución son diferentes para cada tipo de cambio (Elias and Lagergren 2007).

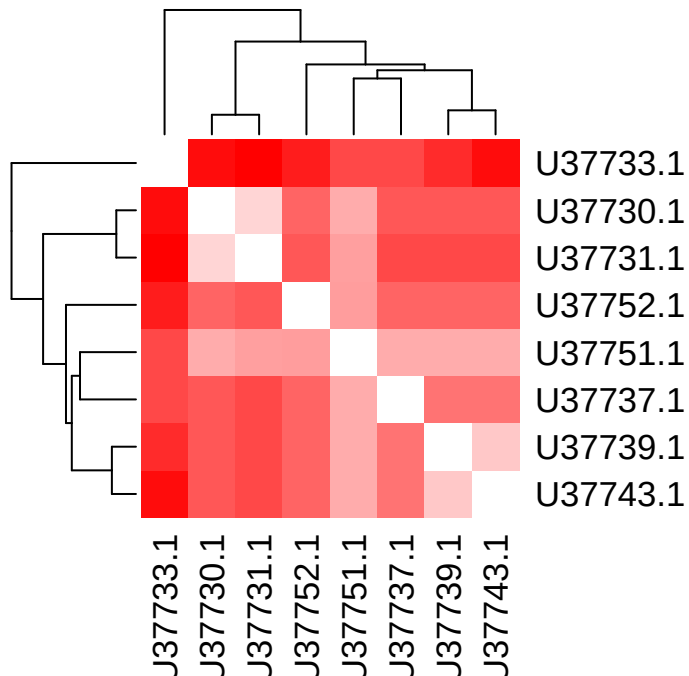


Figure 10.1: Modelo de sustitución de acuerdo a Kimura (1980) o conocido como “Kimura de 2 parámetros”. La probabilidad de transición se define como alfa α mientras que la probabilidad de transversión se define como beta β

Nota: El modelo de Kimura de 2 parámetros es uno de los modelos más simples para calcular distancias genéticas, pero existen otros modelos más complejos que pueden ser utilizados dependiendo del tipo de datos y del análisis que se quiera realizar. Es importante elegir el modelo adecuado para obtener resultados precisos y confiables. Discutirlo con tu tutor(a) es muy importante.

Ahora que tenemos las distancias genéticas, es necesario observar gráficamente las distancias entre secuencias. Podemos hacer esto generando un plot de las distancias genéticas utilizando un plot de calor (heatmap) o un dendrograma. Para generar un heatmap, podemos usar la función `heatmap` del paquete `stats`:

```
# Generamos un heatmap de las distancias genéticas
heatmap(as.matrix(distancias_geneticas), symm = TRUE, col = colorRampPalette(c("white", "red"
```

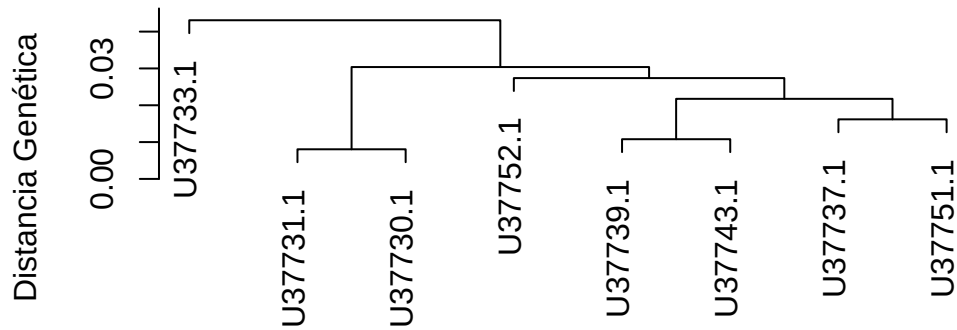


Los colores en el heatmap representan la magnitud de las distancias genéticas entre las secuencias, donde el blanco representa distancias genéticas cercanas a cero (secuencias muy similares) y el rojo representa distancias genéticas mayores (secuencias más diferentes).

Para generar un dendrograma, podemos usar la función `hclust` del paquete `stats`:

```
# Generamos un dendrograma de las distancias genéticas
dendrograma <- hclust(distancias_geneticas, method = "average")
plot(dendrograma, main = "Dendrograma de Distancias Genéticas", xlab = "Secuencias", ylab = "Distancias")
```


Dendrograma de Distancias Genéticas



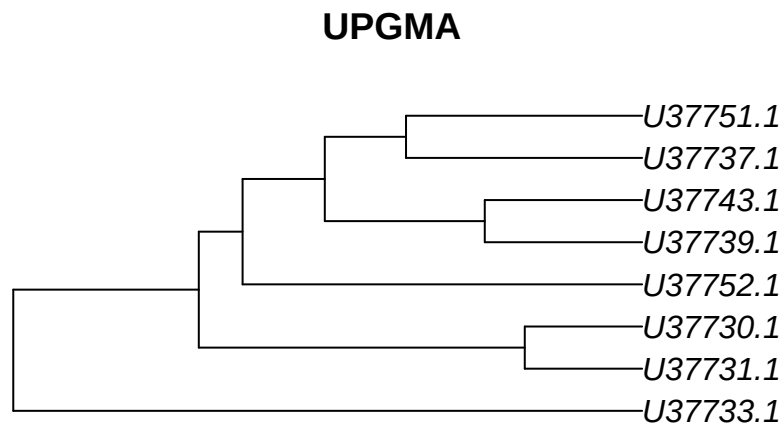
Secuencias
hclust (*, "average")

El dendrograma muestra las relaciones evolutivas entre las secuencias, donde las secuencias que están más cercanas entre sí en el dendrograma tienen distancias genéticas menores, lo que sugiere que están más relacionadas evolutivamente. Las secuencias que están más alejadas entre sí en el dendrograma tienen distancias genéticas mayores, lo que sugiere que están menos relacionadas evolutivamente.

11 Algoritmos de construcción de árboles filogenéticos

Una vez que tenemos las distancias genéticas entre las secuencias, podemos construir un árbol filogenético para visualizar las relaciones evolutivas entre las secuencias. Para esto, podemos usar diferentes algoritmos de construcción de árboles filogenéticos, como el método de UPGMA (Unweighted Pair Group Method with Arithmetic Mean) o el método de Neighbor-Joining. En este caso, usaremos el método de UPGMA para construir el árbol filogenético utilizando la función `upgma` del paquete `phangorn`:

```
# Cargamos el paquete "phangorn" para usar la función upgma
suppressPackageStartupMessages(library(phangorn))
# Construimos el árbol filogenético utilizando el método de UPGMA
tree_upgma <- upgma(distancias_geneticas)
# Visualizamos el árbol filogenético
plot(tree_upgma, main = "UPGMA")
```



Y también podemos usar el método de Neighbor-Joining para construir el árbol filogenético utilizando la función `nj`:

```
# Construimos el árbol filogenético utilizando el método de Neighbor-Joining
tree_nj <- nj(distancias_geneticas)
# Visualizamos el árbol filogenético
plot(tree_nj, main = "Neighbor-Joining")
```

Neighbor-Joining



Hay que recordar que UPGMA asume que las tasas de evolución son constantes a lo largo del tiempo (reloj molecular), lo que puede no ser siempre el caso, mientras que Neighbor-Joining no hace esta suposición y puede ser más adecuado para datos con tasas de evolución variables. Es importante elegir el método adecuado para construir el árbol filogenético dependiendo de las características de los datos y del análisis que se quiera realizar. Discutirlo con tu tutor(a) es muy importante.

Por otra parte, es importante generar un árbol filogenético con soporte estadístico para evaluar la confiabilidad de las relaciones evolutivas entre las secuencias. Para esto, podemos usar el método de bootstrap para generar múltiples réplicas del árbol filogenético y calcular el soporte estadístico para cada nodo del árbol. Para esto, podemos usar la función `bootstrap` del paquete `phangorn`:

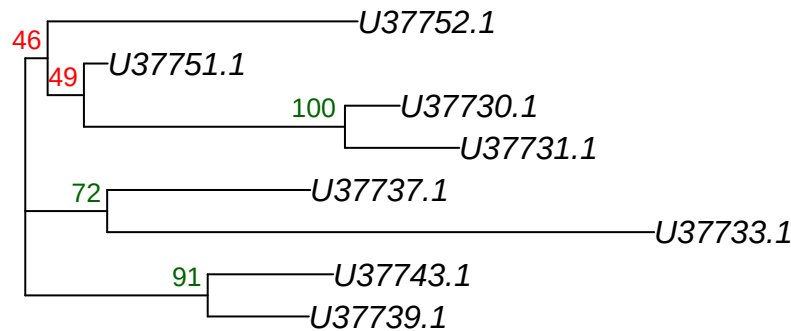
```
fun_nj <- function(x) nj(dist.dna(x, model = "K80"))
# Generamos un árbol filogenético con soporte estadístico utilizando el método de bootstrap
```

```
bs_values <- boot.phylo(tree_nj, alineamiento_dnabin, fun_nj, B = 100)# B es el número de rép
```

```
Running bootstraps:      100 / 100  
Calculating bootstrap values... done.
```

```
# Visualizamos el árbol filogenético con soporte estadístico  
plot(tree_nj, main = "Árbol Final con Soporte de Bootstrap")  
nodelabels(bs_values, adj = c(1.2, -0.5), frame = "n", cex = 0.8,  
           col = ifelse(bs_values > 70, "darkgreen", "red")) # Agregamos los valores de boo
```

Árbol Final con Soporte de Bootstrap



En este código, `boot.phylo` genera múltiples réplicas del árbol filogenético utilizando el método de bootstrap y calcula el soporte estadístico para cada nodo del árbol. Los valores de bootstrap se agregan a los nodos del árbol utilizando la función `nodelabels`, donde los nodos con valores de bootstrap mayores a 70 se colorean en verde oscuro, indicando un soporte estadístico fuerte, mientras que los nodos con valores de bootstrap menores o iguales a 70 se colorean en rojo, indicando un soporte estadístico débil. Es importante interpretar los valores de bootstrap con cautela, ya que un valor de bootstrap alto no garantiza que la relación evolutiva representada por ese nodo sea correcta, pero sí sugiere que esa relación es más confiable que un nodo con un valor de bootstrap bajo.

12 Ejercicio 2:

Recuerda que estos datos deben ser generados a partir de los que descargaste anteriormente. 1. Construye un árbol filogenético utilizando el método de UPGMA y otro utilizando el método de Neighbor-Joining. 2. Compara los árboles filogenéticos generados con los diferentes métodos y discute las diferencias y similitudes entre ellos. 3. Genera un árbol filogenético con soporte estadístico utilizando el método de bootstrap y discute la confiabilidad de las relaciones evolutivas entre las secuencias basándote en los valores de bootstrap obtenidos. 4. Guarda los árboles filogenéticos generados en formato Newick para su posterior análisis. Para esto, puedes usar la función `write.tree` del paquete `ape` para guardar los árboles filogenéticos en formato Newick. Aquí te dejo un ejemplo de cómo hacerlo:

```
# Guardamos el árbol filogenético generado con el método de UPGMA en formato New
write.tree(tree_upgma, file = "../Results/arbol_upgma.newick")
# Guardamos el árbol filogenético generado con el método de Neighbor-Joining en formato Newi
write.tree(tree_nj, file = "../Results/arbol_nj.newick")
```

13 Referencias

14 Summary

In summary, this book has no content whatsoever.

References

(**article?**) {Sayers2022, abstract = {GenBank® (<https://www.ncbi.nlm.nih.gov/genbank/>) is a comprehensive, public database that contains 15.3 trillion base pairs from over 2.5 billion nucleotide sequences for 504 000 formally described species. Recent updates include resources for data from the SARS-CoV-2 virus, including a SARS-CoV-2 landing page, NCBI Datasets, NCBI Virus and the Submission Portal. We also discuss upcoming changes to GI identifiers, a new data management interface for BioProject, and advice for providing contextual metadata in submissions.}, author = {Eric W Sayers and Mark Cavanaugh and Karen Clark and Kim D Pruitt and Conrad L Schoch and Stephen T Sherry and Ilene Karsch-Mizrachi}, doi = {10.1093/nar/gkab1135}, issn = {0305-1048}, issue = {D1}, journal = {Nucleic Acids Research}, month = {1}, pages = {D161-D164}, title = {GenBank}, volume = {50}, url = {<https://doi.org/10.1093/nar/gkab1135>}, year = {2022} }

Elias, Isaac, and Jens Lagergren. 2007. “Fast Computation of Distance Estimators.” *BMC Bioinformatics* 8: 89. <https://doi.org/10.1186/1471-2105-8-89>.

Sayers, Eric W, Mark Cavanaugh, Karen Clark, Kim D Pruitt, Conrad L Schoch, Stephen T Sherry, and Ilene Karsch-Mizrachi. 2022. “GenBank.” *Nucleic Acids Research* 50 (January): D161–64. <https://doi.org/10.1093/nar/gkab1135>.